Seq2seq 与 Attention

对照 Lena Voita NLP 课程的零基础精读

2026 年 4 月 22 日

摘要

本文是对 Lena Voita 的 NLP 课程章节 *Sequence to Sequence (seq2seq) and Attention* 的零基础中文精读。按原文目录顺序，把每一节都拆开重讲：先抛出背景问题，再给朴素直觉，然后写成数学表达，最后给出代码 / 工程对应。读者只需要会基本的线性代数和 softmax 即可入门。

目录

1 序列到序列基础 (Sequence to Sequence Basics) 4

1.1 章节定位与最该问的几个问题 4

1.2 Encoder–Decoder 框架 4

1.3 条件语言模型 (Conditional Language Model) 5

1.3.1 从 LM 到 CLM 的形式化 5

1.3.2 三步流水线 5

1.4 最简单的模型：两个 RNN/LSTM 6

1.4.1 形式化 6

1.4.2 两个值得记住的历史细节 6

1.5 训练：交叉熵损失 (Cross-Entropy Loss) 7

1.5.1 每一步：把真值看作 one-hot 7

1.5.2 整个序列：把每步加起来 7

1.5.3 为什么是这个损失？ 7

1.5.4 Teacher Forcing 与暴露偏差 7

1.6 推理：贪心解码与 Beam Search 8

1.6.1 方案 A：贪心解码 (Greedy Decoding) 8

1.6.2 方案 B：Beam Search 8

1.6.3 Beam 越大就越好吗？——并不 9

1.7 这一章真正该记住的 10

2 Attention：让解码器自己去原文里取信息 10

2.1 章节定位 + 零基础该问的问题 10

2.2 固定编码器表示的瓶颈 11

2.2.1 背景问题：一个向量装不下整句话 11

2.2.2 朴素直觉：把宇宙压进 512 个数 . . . 11
2.2.3 数学表达：瓶颈到底在哪一项 . . . 11
2.3 Attention 的高层视角 . . . 11
2.3.1 背景问题：能不能让解码器“回头看”? . . . 11
2.3.2 朴素直觉：加权查表 . . . 12
2.3.3 数学表达：attention 的三步公式 . . . 12
2.4 怎么算 attention score? . . . 13
2.4.1 背景问题：一个 score 函数能管所有情况吗 . . . 13
2.4.2 朴素直觉：从“纯几何”到“多塞一层 MLP” . . . 13
2.4.3 数学表达：三种常见 score 函数 . . . 13
2.5 模型变体：Bahdanau vs Luong . . . 14
2.5.1 背景问题：attention 应该插在解码器的哪一步 . . . 14
2.5.2 朴素直觉：时序对比 . . . 14
2.5.3 数学表达：两个版本的关键公式 . . . 14
2.6 Attention 学到了（近似的）对齐 . . . 15
2.6.1 背景问题：这些 α_{tk} 真的有意义吗 . . . 15
2.6.2 朴素直觉：软对齐 . . . 15
2.6.3 从可解释性的角度看 . . . 15
2.7 这一章真正该记住的 . . . 16

3 Transformer：Attention is All You Need 16
3.1 章节定位 . . . 16
3.2 自注意力 Self-Attention：让 token 互相看 . . . 17
3.3 Masked Self-Attention：decoder 不许偷看未来 . . . 18
3.4 多头注意力 Multi-Head Attention . . . 19
3.5 Transformer 模型架构 . . . 20
3.5.1 Feed-Forward Block (FFN) . . . 20
3.5.2 残差连接 Residual Connections . . . 20
3.5.3 层归一化 Layer Normalization . . . 21
3.5.4 位置编码 Positional Encoding . . . 21
3.5.5 把整体拼起来 . . . 22
3.6 这一章真正该记住的 . . . 22

4 子词切分：字节对编码 BPE 23
4.1 章节定位与零基础该问的问题 . . . 23
4.2 固定词表问题 . . . 23
4.3 BPE 算法的两个阶段 . . . 24
4.3.1 训练阶段：学合并规则 . . . 24
4.3.2 推理阶段：按合并表切新词 . . . 25
4.4 一些值得知道的工程细节 . . . 26
4.5 这一章真正该记住的 . . . 26

5 分析与可解释性 (Analysis and Interpretability) 26
5.1 章节定位 + 零基础该问的问题 26
5.2 多头自注意力的头到底在干嘛? 27
5.2.1 重要的 head 是可解释的 27
5.2.2 大多数 head 可以被剪掉 27
5.3 Probing：表示里到底编码了什么? 28
5.3.1 Probing 的方法论 28
5.3.2 NMT encoder 学到了什么形态学? 28
5.4 这一章真正该记住的 29

致谢与原文 29

1 序列到序列基础 (Sequence to Sequence Basics)

1.1 章节定位与最该问的几个问题

很多任务的输入和输出都是长度可变的序列：把一句英文翻译成中文、把一段长文压缩成摘要、把一段 Python 代码翻译成等价的 C++、甚至把一张图片描述成一句话——这些都属于 *序列到序列* (sequence to sequence, seq2seq) 任务。它们共享同一个抽象：给定源序列 $\boldsymbol{x} = (x_1, \ldots, x_S)$，要生成目标序列 $\boldsymbol{y} = (y_1, \ldots, y_T)$，而 S 和 T 通常不相等、也不预先固定。

背景问题

在正式进入公式之前，先把本章想回答的几个朴素问题列出来。读完这一章你应该可以回答：

1. Encoder 和 decoder 分别是干什么的？为什么要拆成两块？
2. “条件语言模型” (conditional language model, CLM) 跟普通的语言模型差在哪？
3. 训练时为什么用交叉熵损失？它和最大似然是同一件事吗？
4. 训练时 decoder 看到的“上一个词”到底是真值还是模型自己的预测？为什么这么做？
5. 推理 (inference) 时为什么 *greedy* 不够，要用 *beam search*？
6. Beam 越大就越好吗？

本章只覆盖最朴素的 seq2seq 框架（两个 RNN 之间用一个固定向量传话）。这个框架有一个明显的短板：信息全靠一个固定向量瓶颈，长句子会“记不住”。第二章引入的 **attention** 正是为了解决这个瓶颈。所以请把这一章当成“为什么我们后面非得有 attention 不可”的铺垫。

1.2 Encoder–Decoder 框架

朴素直觉

最朴素的直觉：既然输入输出都是序列，而且通常长度不同，那就分两步走。

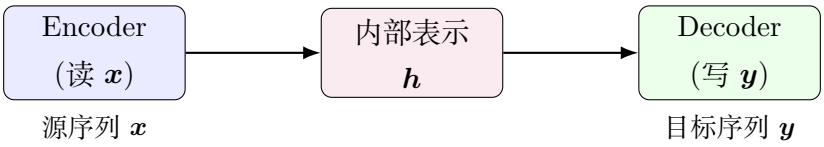
- **Encoder**（编码器）：把源序列 $\boldsymbol{x}$ 读完，压缩成某种“它到底在说什么”的内部表示。
- **Decoder**（解码器）：拿到这个表示，一个词一个词地写出目标序列 $\boldsymbol{y}$。

为什么这种“先读后写”的拆分如此自然？

- 输入和输出长度通常不同，无法用单一前馈网络一一对应。
- 在写第一个目标词之前，最好已经看完整个源句——否则连主语是谁都还不知道。
- 拆开之后，encoder 和 decoder 可以使用不同的网络结构 (RNN、CNN、Transformer 都可以)，便于实验和组合。

备注 1.1. *此后所有更复杂的模型——加了 attention 的 RNN、Transformer、甚至现代 LLM 中的 encoder-only / decoder-only 变体——都没有抛弃这个骨架，而只是在改“encoder 怎么编”、“decoder 怎么用编码出来的东西”这两件事。*

下面这张极简的“三盒图”足够你在脑子里画：



1.3 条件语言模型 (Conditional Language Model)

朴素直觉

普通的语言模型 (LM) 干的事是：在没有任何外部输入的情况下，告诉你下一个词最可能是什么。条件语言模型多了一个“条件”——多给它一个 $\boldsymbol{x}$，它再预测下一个词。Seq2seq 的 decoder 就是这样一个条件语言模型，条件是源序列。

1.3.1 从 LM 到 CLM 的形式化

普通语言模型把一句话的概率拆成左到右的链式乘积：

$$p(\boldsymbol{y}) \;=\; \prod_{t=1}^{T} p(y_t \mid y_{<t}), \qquad y_{<t} := (y_1, \ldots, y_{t-1}).$$

Seq2seq 模型做的事完全一样，只是每一步都额外条件于源序列 $\boldsymbol{x}$：

条件语言模型 (CLM) 的核心公式

$$\boxed{p(\boldsymbol{y} \mid \boldsymbol{x}) \;=\; \prod_{t=1}^{T} p(y_t \mid y_{<t}, \boldsymbol{x})}$$

也就是说：*seq2seq* = 条件语言模型。它和普通 LM 在结构上几乎一样，差别只在“写每个词的时候，模型可以额外看一眼 $\boldsymbol{x}$”。

1.3.2 三步流水线

不论 encoder/decoder 内部用什么神经网络，每一步生成 y_t 都是同样三步：

1. **喂入**：把源序列 $\boldsymbol{x}$ 和已生成的前缀 $y_{<t}$ 一起送入网络。
2. **得到表示**：网络给出一个 d 维向量 $\boldsymbol{h}_t \in \mathbb{R}^d$，代表“此刻应该写什么样的词”。
3. **打分并归一化**：经过一个线性层 $W \in \mathbb{R}^{|V| \times d}$ 把 $\boldsymbol{h}_t$ 映射到词表 V 上，再用 softmax 得到下一个词的分布：

$$p(y_t \mid y_{<t}, \boldsymbol{x}) \;=\; \mathrm{softmax}(W\boldsymbol{h}_t)_{y_t}.$$

备注 1.2. *$\boldsymbol{x}$ 不必是文本。把图像先用 CNN 编码成向量，再让 decoder 写出一段描述，就是 image captioning。所以 CLM 比“文本到文本的 seq2seq”更广泛——它是一种通用模板。*

代码 / 工程对应

代码骨架（伪代码）：

```
h_src = encoder(x) # 源序列的表示
y_prev = [BOS]
```

```
for t in range(T_max):
    h_t = decoder(y_prev, h_src) # 第 t 步的隐表示
    logits = W @ h_t # 线性投影到 |V|
    probs = softmax(logits) # 下一个词的分布
    y_t = sample_or_argmax(probs)
    y_prev.append(y_t)
    if y_t == EOS: break
```

1.4 最简单的模型：两个 RNN/LSTM

朴素直觉

既然要“读完 $\boldsymbol{x}$ 再写 $\boldsymbol{y}$”，最朴素的实现就是：用一个 RNN 把源序列从左到右扫一遍，把它的最后一个隐藏状态当成“整句话的总结”，再把这个状态作为另一个 RNN 的初始状态，让它一个词一个词写出目标序列。

1.4.1 形式化

设 encoder 的隐状态序列为 $\boldsymbol{h}_1^{\text{enc}}, \ldots, \boldsymbol{h}_S^{\text{enc}}$，decoder 的隐状态序列为 $\boldsymbol{h}_1^{\text{dec}}, \ldots, \boldsymbol{h}_T^{\text{dec}}$。最朴素的 seq2seq 模型只用一根细线把两者连起来：

$$\boldsymbol{h}_s^{\text{enc}} = \text{RNN}_{\text{enc}}(\boldsymbol{h}_{s-1}^{\text{enc}}, x_s), \qquad s = 1, \ldots, S, \tag{1}$$

$$\boldsymbol{h}_0^{\text{dec}} = \boldsymbol{h}_S^{\text{enc}}, \tag{2}$$

$$\boldsymbol{h}_t^{\text{dec}} = \text{RNN}_{\text{dec}}(\boldsymbol{h}_{t-1}^{\text{dec}}, y_{t-1}), \qquad t = 1, \ldots, T. \tag{3}$$

关键就在式 (2)：源句的全部信息——长度 S 任意——都被压成 一个固定维度的向量 $\boldsymbol{h}_S^{\text{enc}}$ 传给 decoder。

记住这一点

这一根细线就是固定向量瓶颈 (fixed-size bottleneck)。它在短句上工作良好，但句子越长，单个向量越塞不下，翻译质量会随句长显著下降。**这正是第二章 attention 要解决的问题**。

1.4.2 两个值得记住的历史细节

备注 1.3 (Sutskever 等人 2014)**.** *最早把“两个 LSTM 堆起来做翻译”这套打通的工作是 Sutskever, Vinyals, Le 在 2014 年的* Sequence to Sequence Learning with Neural Networks。*它在 WMT 英法翻译上首次让纯神经方法逼近当时主流的统计机器翻译系统。*

备注 1.4 (反转源句的小技巧)**.** *他们还报告了一个反直觉但很有效的 trick：把源句倒着喂给 encoder。这样源句开头的词（往往对应目标句开头的词）和 decoder 第一步在时间上更接近，梯度路径更短，优化更顺利。这个技巧本身在 attention 出现后基本被遗忘，但它揭示了瓶颈结构对“距离”非常敏感。*

1.5 训练：交叉熵损失 (Cross-Entropy Loss)

背景问题

模型每一步吐出来的是一个长度为 $|V|$ 的概率分布 $p(y_t \mid y_{<t}, \boldsymbol{x})$。而真实的下一个词 y_t^* 只是词表上的某一个。怎么把“分布”和“某个词”比出一个损失？

1.5.1 每一步：把真值看作 one-hot

把真实的下一个词 y_t^* 写成 one-hot 向量 $\boldsymbol{e}_{y_t^*} \in \{0,1\}^{|V|}$：只有第 y_t^* 个分量是 1，其余为 0。模型的预测分布记为 $\boldsymbol{p}_t \in \Delta^{|V|-1}$。两者之间的交叉熵定义为

$$H(\boldsymbol{e}_{y_t^*}, \boldsymbol{p}_t) \;=\; -\sum_{w \in V} \boldsymbol{e}_{y_t^*}[w] \log \boldsymbol{p}_t[w] \;=\; -\log p(y_t^* \mid y_{<t}, \boldsymbol{x}).$$

因为 one-hot 只有一个位置非零，整个求和坍缩成单项：就是模型给真值那个词的负对数概率。

1.5.2 整个序列：把每步加起来

Seq2seq 的训练损失

$$\mathcal{L}(\boldsymbol{x}, \boldsymbol{y}^*) \;=\; -\sum_{t=1}^{T} \log p(y_t^* \mid y_{<t}^*, \boldsymbol{x}).$$

在一个 mini-batch 上，对所有句子再取平均（或先除以各自长度后再取平均）。

1.5.3 为什么是这个损失？

命题 1.5 (交叉熵 = 最大似然 = 最小 KL)**.** 最小化上面这个交叉熵损失，等价于在训练数据上做最大似然估计：

$$\min_{\theta} \; -\sum_{(\boldsymbol{x},\boldsymbol{y}^*)} \log p_\theta(\boldsymbol{y}^* \mid \boldsymbol{x}) \iff \max_{\theta} \prod_{(\boldsymbol{x},\boldsymbol{y}^*)} p_\theta(\boldsymbol{y}^* \mid \boldsymbol{x}),$$

也等价于让模型分布 $p_\theta(\cdot \mid \boldsymbol{x})$ 在 KL 意义下尽量逼近数据经验分布 $\hat{p}_{data}(\cdot \mid \boldsymbol{x})$。

1.5.4 Teacher Forcing 与暴露偏差

训练时还有一个常常被新手忽略的细节：在算第 t 步的损失时，decoder 的输入“上一个词”是用真值 y_{t-1}^*，不是模型自己上一步预测出来的词。这种做法叫做 **teacher forcing**（教师强迫）。

- **为什么这么做？**训练初期模型预测几乎全错，如果一直拿自己的错答案当输入，后面所有时间步都建立在错误前缀上，损失曲面非常崎岖、根本学不动。用真值前缀可以让每一步都在“合理”的上下文里学习，训练快很多。

- **副作用：暴露偏差 (exposure bias)**。推理时没有真值可用，模型只能看自己之前的预测；而训练时它从来没见过自己的错误前缀。这种训练-推理不一致会让长序列生成时错误累积。后续课程里 scheduled sampling、minimum risk training、RL 微调等都是为了缓解这件事，但本章先放一放。

代码 / 工程对应

训练循环骨架（伪代码）：

```
h_src = encoder(x)
h_dec = h_src # 瓶颈传话
loss = 0
for t in range(1, T+1):
    h_dec = decoder_step(h_dec, y_star[t-1]) # teacher forcing
    logits = W @ h_dec
    loss += cross_entropy(logits, y_star[t]) # = -log p(y*_t | ...)
loss.backward()
```

1.6 推理：贪心解码与 Beam Search

背景问题

训练好之后，给定一个新的 $\boldsymbol{x}$，我们想要的是最可能的目标序列：

$$\boldsymbol{y}^\star \;=\; \arg\max_{\boldsymbol{y}}\; p(\boldsymbol{y} \mid \boldsymbol{x}).$$

但词表大小 $|V|$ 通常上万，目标长度可达几十——可能的 $\boldsymbol{y}$ 有 $|V|^T$ 种，穷举绝对不可行。怎么办？

1.6.1 方案 A：贪心解码 (Greedy Decoding)

朴素直觉

最朴素的近似：每一步都挑当前条件概率最大的那个词，写下来，作为下一步的输入，反复直到 EOS。

$$\hat{y}_t \;=\; \arg\max_{w \in V}\; p(w \mid \hat{y}_{<t}, \boldsymbol{x}).$$

优点是快、实现极简单；缺点是局部最优不等于全局最优。某一步选了概率第一的词，可能让后续所有步骤都陷入低概率区域；如果当时选了概率第二的词，整句话反而更可能。

例 1.6. 英译中：源句 “I saw her duck.”。*Greedy* 在第三步可能锁定 “鸭子”，导致整句翻成 “我看见她的鸭子”；但若回退一步把 duck 当动词，整句应是 “我看见她躲开” ——后者作为整句的概率更高，却被 *greedy* 永远错过。

1.6.2 方案 B：Beam Search

朴素直觉

不要在每一步只留一个候选，而是始终保留 N 个最有希望的部分序列（叫 beam）。每一步把这 N 个候选各扩展 $|V|$ 种延续，得到 $N \cdot |V|$ 个新候选，再按累计 log 概率挑出前 N 个，进入下一步。

Beam Search 的打分

对一条部分假设 $\boldsymbol{y}_{1:t}$，累计得分是 log 概率之和：

$$\mathrm{score}(\boldsymbol{y}_{1:t} \mid \boldsymbol{x}) \;=\; \sum_{i=1}^{t} \log p(y_i \mid y_{<i}, \boldsymbol{x}).$$

每步保留 score 最高的 N 条。N 称为 *beam size*（束宽），机器翻译里常取 $N = 4 \sim 10$。

- $N = 1$ 时退化为 greedy。
- $N \to \infty$ 时趋近于全局最优搜索（穷举）。

1.6.3 Beam 越大就越好吗？——并不

经验上，beam size 不是越大越好：太大反而会让翻译质量下降。常见的解释有两类：

- **长度偏置 (length bias)**。log 概率是负数，每加一个词都让总分变小，所以 beam 越大、越倾向于挑更短的句子，可能会丢掉应有的内容。常用的修补是长度归一化 (length normalization)：用 $\mathrm{score}(\boldsymbol{y}_{1:t})/t^{\alpha}$（$\alpha \in [0.6, 1]$）作为打分，抵消长度对累计 log 概率的拉低。
- **模型本身的过自信**。训练时只见过真实序列，模型把概率质量过度集中在某些“典型”句子上；当 beam 足够大时，搜索算法会“恰好”找到一些极短或退化的高分序列（例如直接输出 EOS）。这是模型问题，不是搜索问题。

代码 / 工程对应

Beam search 的最小骨架（伪代码）：

```
beams = [(["BOS"], 0.0)] # (前缀, 累计 log 概率)
for t in range(T_max):
    candidates = []
    for prefix, score in beams:
        if prefix[-1] == "EOS":
            candidates.append((prefix, score))
            continue
        logp = log_softmax(decoder_step(prefix, x)) # 形状 (|V|,)
        for w, lp in topk(logp, k=N): # 只看每条的 top-N 延续
            candidates.append((prefix + [w], score + lp))
    beams = topk_by_score(candidates, k=N) # 全局再挑前 N
return best_by_length_normalized_score(beams)
```

1.7 这一章真正该记住的

记住这一点

1. **Seq2seq = encoder + decoder**。一边读源序列、一边写目标序列；后续所有更复杂的模型都没有抛弃这个骨架，只是在改两边怎么实现、怎么连接。
2. **Decoder 是一个条件语言模型**：$p(\boldsymbol{y} \mid \boldsymbol{x}) = \prod_t p(y_t \mid y_{<t}, \boldsymbol{x})$。它和普通 LM 唯一的区别就是额外条件于 $\boldsymbol{x}$。
3. **最朴素实现 = 两个 RNN + 一个固定向量瓶颈**。这个瓶颈对长句不友好，正是第二章 attention 要解决的痛点。
4. **训练用 token 级交叉熵**，等价于对训练序列做最大似然。Decoder 输入用真值前缀 (*teacher forcing*)，会带来训练-推理不一致 (*exposure bias*)。
5. **推理是搜索 $\arg\max_{\boldsymbol{y}} p(\boldsymbol{y} \mid \boldsymbol{x})$ 的近似**。Greedy 局部最优、Beam search 用宽度为 N 的近似全局搜索，常用 $N \approx 4$–10。
6. **Beam 不是越大越好**，原因是长度偏置和模型过自信；长度归一化是最常见的实用修补。

2 Attention：让解码器自己去原文里取信息

2.1 章节定位 + 零基础该问的问题

上一章我们看到，经典的 seq2seq 把整句源语言压成一个向量 $\boldsymbol{h}_m^{\text{enc}}$，再交给解码器。这一章要回答的问题是：当这个“压缩到一个向量”的方案吃力的时候，模型应该怎么改？答案就是 *attention*。它是 Transformer 之前 NMT 时代最重要的进展，也是 self-attention 的直接祖先。

背景问题

[一上来该问的几个问题] 带着这些问题往下读，比一上来就背公式更划算：

1. RNN 编码器到底卡在哪？为什么句子一长就翻不动？
2. Attention 的输入是什么、输出是什么？它放在 seq2seq 的哪个位置？
3. Attention score 函数有几种，互相差在哪？
4. Bahdanau attention 和 Luong attention 是同一个东西吗？哪里不一样？
5. 学出来的 attention 权重，为什么大家都说它“像对齐”？这有什么用？

2.2 固定编码器表示的瓶颈

2.2.1 背景问题：一个向量装不下整句话

背景问题

经典 seq2seq 的编码器读完全句之后，吐出一个固定维度的向量 $\boldsymbol{h}_m^{\text{enc}} \in \mathbb{R}^d$，把它当作整句的“意思”交给解码器。但是：

- 句子是 5 个词还是 50 个词，最后给解码器的都是同一个 d 维向量。
- 解码器在生成第 1 个目标词时关心的源信息，和生成第 10 个目标词时关心的源信息，显然不一样，但它每一步看到的“源信息”都是同一个向量。

2.2.2 朴素直觉：把宇宙压进 512 个数

朴素直觉

Lena 在博客里有一个非常形象的比喻：让编码器把整句话压成一个固定向量，就像让你把整个宇宙压进 *512* 个数里再交出去。短句还行，长句就一定会丢信息。
更糟的是：解码器每一步都要从同一个“宇宙摘要”里翻出当前需要的那一小块信息——它根本没机会“回头再看一眼原文”。

2.2.3 数学表达：瓶颈到底在哪一项

固定编码器表示的损失

经典 seq2seq 的解码概率分解为

$$p(\boldsymbol{y} \mid \boldsymbol{x}) \;=\; \prod_{t=1}^{n} p(y_t \mid y_{<t}, \boldsymbol{h}_m^{\text{enc}}) \,. \tag{4}$$

注意右边条件里的 $\boldsymbol{h}_m^{\text{enc}}$ 与 t 无关。也就是说，模型被强制要求：

$$\text{对所有解码步}t\text{，源端信息都用同一个向量}\boldsymbol{h}_m^{\text{enc}}\text{ 表达。} \tag{5}$$

当 m 增大时，这个向量必须承担越来越多内容，信息被覆盖、被冲掉。

备注 2.1. *经验上，普通 seq2seq 在源句长度超过 20–30 之后 BLEU 会明显下降；引入 attention 之后这一掉点会被显著拉平。这就是 Bahdanau 等人 2014 年那篇论文最早展示的结果。*

2.3 Attention 的高层视角

2.3.1 背景问题：能不能让解码器“回头看”？

背景问题

既然“压成一个向量”是瓶颈，那很自然的想法是：

- 让编码器把每个时刻的隐状态 $\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m$ 都留下来，不要扔。
- 解码器在第 t 步，根据当前自己的状态 $\boldsymbol{h}_t$，自己决定该重点看源端的哪些位置。

关键约束是：这个“决定看哪里”的过程必须可微，否则没法用反向传播一起训练。

2.3.2 朴素直觉：加权查表

朴素直觉

把编码器输出 $\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m$ 想成一张“源端记忆表”。解码器每一步做的事就是一次软查表：

1. 拿出一个“查询” $\boldsymbol{h}_t$（解码器当前状态）。
2. 对每条记忆 $\boldsymbol{s}_k$ 打一个相关性分数 e_{tk}。
3. 把分数过一个 softmax 得到“看每条记忆的比例” α_{tk}。
4. 按比例把所有记忆加权求和，得到这一步的“context” $\boldsymbol{c}_t$。

注意整个过程没有 argmax、没有硬选择，所以处处可导，可以端到端训练。模型自己会学出“在第 t 步该看哪里”，不需要我们提供人工对齐。

2.3.3 数学表达：attention 的三步公式

解码第 t 步的 attention

设编码器输出为 $\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m \in \mathbb{R}^{d_s}$，解码器当前状态为 $\boldsymbol{h}_t \in \mathbb{R}^{d_h}$。

$$e_{tk} = \mathrm{score}(\boldsymbol{h}_t, \boldsymbol{s}_k), \qquad k = 1, \ldots, m, \tag{6}$$

$$\alpha_{t\cdot} = \mathrm{softmax}(e_{t\cdot}) \iff \alpha_{tk} = \frac{\exp(e_{tk})}{\sum_{j=1}^{m} \exp(e_{tj})}, \tag{7}$$

$$\boldsymbol{c}_t = \sum_{k=1}^{m} \alpha_{tk}\, \boldsymbol{s}_k. \tag{8}$$

得到的 $\boldsymbol{c}_t$ 就是“给第 t 步用的源端摘要”，它会被喂回解码器（拼接到输入、或拼接到隐状态、或参与最终输出层）。

定义 2.2 (Attention 权重)**.** 向量 $\boldsymbol{\alpha}_{t\cdot} = (\alpha_{t1}, \ldots, \alpha_{tm})$ 满足 $\alpha_{tk} \geq 0$ 且 $\sum_k \alpha_{tk} = 1$，称为解码第 t 步对源端的 attention 权重。它描述了“第 t 步用了源端哪些位置、各占多少”。

代码 / 工程对应

工程上要记住三件事：

- 编码器需要把所有时刻的 $\boldsymbol{s}_k$ 都缓存下来（一个 $m \times d_s$ 矩阵），而不是只留最后一个。
- 每个解码步要做一次 $O(m)$ 的打分 + softmax + 加权和；总成本相对原 seq2seq 增加了一项 $O(nm)$。
- $\boldsymbol{c}_t$ 怎么“喂回”解码器，是 Bahdanau 与 Luong 两派的主要差异（见 §2.5）。

2.4 怎么算 attention score？

2.4.1 背景问题：一个 score 函数能管所有情况吗

背景问题

公式 (6) 里的 $\mathrm{score}(\boldsymbol{h}_t, \boldsymbol{s}_k)$ 只是一个抽象记号。它要满足两点：

- 输入两个向量，输出一个标量分数；
- 是 $\boldsymbol{h}_t$ 和 $\boldsymbol{s}_k$ 的可微函数。

满足这两点的函数有一堆，历史上常用的就是下面三种。

2.4.2 朴素直觉：从“纯几何”到“多塞一层 MLP”

朴素直觉

- **点积**：直接看两个向量的余弦相似度（按模长缩放就成了后来的 scaled dot-product）。最便宜，但要求 $\boldsymbol{h}_t$ 和 $\boldsymbol{s}_k$ 在同一空间里有意义。
- **双线性 (bilinear)**：在中间塞一个矩阵 $\boldsymbol{W}$，相当于先把其中一个向量投影到对方的空间。$\boldsymbol{h}_t$ 和 $\boldsymbol{s}_k$ 维度不同也能用。
- **加性 (additive / MLP)**：把两个向量先各自线性变换、加起来，再过 tanh，再投到一个标量。最“能学”，但参数和算力都最贵。

2.4.3 数学表达：三种常见 score 函数

三种 attention score

$$\textbf{点积 (dot)}: \quad \mathrm{score}(\boldsymbol{h}, \boldsymbol{s}) = \boldsymbol{h}^\top \boldsymbol{s}, \tag{9}$$

$$\textbf{双线性 (Luong)}: \quad \mathrm{score}(\boldsymbol{h}, \boldsymbol{s}) = \boldsymbol{h}^\top \boldsymbol{W} \boldsymbol{s}, \tag{10}$$

$$\textbf{加性 (Bahdanau)}: \quad \mathrm{score}(\boldsymbol{h}, \boldsymbol{s}) = \boldsymbol{v}^\top \tanh(\boldsymbol{W}_1 \boldsymbol{h} + \boldsymbol{W}_2 \boldsymbol{s}). \tag{11}$$

其中 $\boldsymbol{W} \in \mathbb{R}^{d_h \times d_s}$，$\boldsymbol{W}_1 \in \mathbb{R}^{d \times d_h}$，$\boldsymbol{W}_2 \in \mathbb{R}^{d \times d_s}$，$\boldsymbol{v} \in \mathbb{R}^d$ 都是可学习参数。

备注 2.3 (谁更好用).

- **速度**：点积 $\gg$ 双线性 $>$ 加性。点积可以一次矩阵乘法批量算完，所以后来 *Transformer* 选了它。
- **表达力**：加性最强（多了一层非线性），点积最弱。
- **兼容性**：维度不一致时只能用双线性或加性。

2.5 模型变体：Bahdanau vs Luong

2.5.1 背景问题：attention 应该插在解码器的哪一步

背景问题

有了 attention 模块以后，下一个问题是：

- 解码器更新隐状态的之前就先算 $\boldsymbol{c}_t$，把它和输入一起喂给 RNN？还是
- 让 RNN 先正常更新出 $\boldsymbol{h}_t$，再用 $\boldsymbol{h}_t$ 去算 $\boldsymbol{c}_t$，最后把两者合起来出预测？

这就是 Bahdanau (2014) 和 Luong (2015) 两派的核心区别。

2.5.2 朴素直觉：时序对比

朴素直觉

两条时序线，一眼对比：

- **Bahdanau**：$\boldsymbol{h}_{t-1} \longrightarrow$ attention $\longrightarrow \boldsymbol{c}_t \longrightarrow$ RNN 用$(\boldsymbol{c}_t, y_{t-1})$ 算$\boldsymbol{h}_t \longrightarrow$ 预测y_t。
  Attention 在“解码 RNN 一步”之前发生。
- **Luong**：$\boldsymbol{h}_{t-1} \longrightarrow$ RNN 用y_{t-1} 算$\boldsymbol{h}_t \longrightarrow$ attention$(\boldsymbol{h}_t) \longrightarrow \boldsymbol{c}_t \longrightarrow \tilde{\boldsymbol{h}}_t = \tanh(\boldsymbol{W}[\boldsymbol{h}_t; \boldsymbol{c}_t]) \longrightarrow$ 预测y_t。
  Attention 在“解码 RNN 一步”之后发生。

2.5.3 数学表达：两个版本的关键公式

Bahdanau attention（2014）

- 编码器是双向 RNN：$\boldsymbol{s}_k = [\overrightarrow{\boldsymbol{s}}_k; \overleftarrow{\boldsymbol{s}}_k]$。
- Score 用加性：$e_{tk} = \boldsymbol{v}^\top \tanh(\boldsymbol{W}_1 \boldsymbol{h}_{t-1} + \boldsymbol{W}_2 \boldsymbol{s}_k)$。
- 注意 query 是上一步的 $\boldsymbol{h}_{t-1}$，先算 $\boldsymbol{c}_t$，再更新：

$$e_{tk} = \boldsymbol{v}^\top \tanh(\boldsymbol{W}_1 \boldsymbol{h}_{t-1} + \boldsymbol{W}_2 \boldsymbol{s}_k), \tag{12}$$

$$\alpha_{t\cdot} = \mathrm{softmax}(e_{t\cdot}), \quad \boldsymbol{c}_t = \sum_k \alpha_{tk} \boldsymbol{s}_k, \tag{13}$$

$$\boldsymbol{h}_t = \mathrm{RNN}(\boldsymbol{h}_{t-1},\ [\boldsymbol{E}\, y_{t-1}; \boldsymbol{c}_t]), \tag{14}$$

$$p(y_t \mid y_{<t}, \boldsymbol{x}) = \mathrm{softmax}(\boldsymbol{W}_o[\boldsymbol{h}_t; \boldsymbol{c}_t; \boldsymbol{E}\, y_{t-1}]). \tag{15}$$

Luong attention（2015，global 版本）

- 编码器是单向 RNN（更简单）。
- Score 用双线性：$e_{tk} = \boldsymbol{h}_t^\top \boldsymbol{W} \boldsymbol{s}_k$。
- Query 是当前步的 $\boldsymbol{h}_t$，先更新 RNN，再算 attention：

$$\boldsymbol{h}_t = \mathrm{RNN}(\boldsymbol{h}_{t-1},\ \boldsymbol{E}\, y_{t-1}), \tag{16}$$

$$e_{tk} = \boldsymbol{h}_t^\top \boldsymbol{W} \boldsymbol{s}_k, \quad \alpha_{t\cdot} = \mathrm{softmax}(e_{t\cdot}), \quad \boldsymbol{c}_t = \sum_k \alpha_{tk} \boldsymbol{s}_k, \tag{17}$$

$$\tilde{\boldsymbol{h}}_t = \tanh(\boldsymbol{W}_c[\boldsymbol{h}_t; \boldsymbol{c}_t]), \tag{18}$$

$$p(y_t \mid y_{<t}, \boldsymbol{x}) = \mathrm{softmax}\Big(\boldsymbol{W}_o \tilde{\boldsymbol{h}}_t\Big). \tag{19}$$

备注 2.4. *Luong 还提出过一个 local attention 版本，只在源端附近一个窗口里算 softmax。但工业界后来还是 global 版本 + 各种改进用得多。*

代码 / 工程对应

理解这两派的差异比记公式重要：

- **时序**：Bahdanau 是“先看再走”，Luong 是“先走再看”。
- **Score**：Bahdanau 用 MLP（参数最多、最能学）；Luong 默认用双线性（更轻）。
- **编码器**：Bahdanau 双向（信息更全），Luong 单向（实现更简单）。
- 这两个模型在 Transformer 出现前统治了神经机器翻译，几乎所有后续 NMT 工程都是它们的变体。

2.6 Attention 学到了（近似的）对齐

2.6.1 背景问题：这些 α_{tk} 真的有意义吗

背景问题

我们没有给模型任何“哪个目标词对应哪个源词”的标注，只是用最大似然让它去预测下一个目标词。可学完之后，把 α_{tk} 画成热力图，往往会看到：解码到目标位置 t 时，权重恰好集中在源端那几个“正在被翻译”的词上。

2.6.2 朴素直觉：软对齐

朴素直觉

统计机器翻译时代，“对齐 (alignment)”是一个核心概念，要靠 IBM Model 1/2 等专门的算法去估。Attention 把这件事顺手做了：

- 每个目标词 y_t 对应一个源端的概率分布 $\boldsymbol{\alpha}_{t\cdot}$。
- 这就是“软对齐”——不再是 hard 的“词 i 对应词 j”，而是“词 t 用了源端 30% 的位置 j_1，60% 的位置 j_2，…”。

2.6.3 从可解释性的角度看

备注 2.5 (Attention 作为归纳偏置)**.** *Bahdanau 那篇论文里展示的 attention 热力图，是深度学习历史上第一批看起来很“可解释”的内部状态：*

- *它没有被强行训练成对齐，但自发地呈现出对齐的样子；*

- 这说明“对每一步只看相关的几条记忆”是一种有用且自然的归纳偏置；
- 这一思想被 *Transformer* 推到极致：所有信息流都通过 *attention* 完成。

不过要小心：后续研究（如 *Jain & Wallace 2019*）也指出，*attention* 权重并不一定等于“真正的解释”。它看起来像对齐，并不代表模型一定用它做决策。

2.7 这一章真正该记住的

记住这一点

1. **病根**：经典 seq2seq 把整句源语言压成一个固定向量；句子一长，就装不下。
2. **治法**：编码器保留所有时刻的状态 $\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m$；解码器每一步对它们做加权查表，权重由 softmax 给出。
3. **三步范式**：score → softmax → 加权和；这套流程全程可导，端到端训练，不需要人工对齐。
4. **Score 三选一**：点积（最快）、双线性（Luong）、加性 MLP（Bahdanau）；速度 vs 表达力的取舍。
5. **Bahdanau vs Luong**：核心差异是“attention 在 RNN 更新之前还是之后”，以及 score 函数和编码器方向；它们是 Transformer 之前 NMT 的两大支柱。
6. **学出来像对齐**：α_{tk} 自发呈现软对齐，是“attention 作为归纳偏置”的第一个直观证据；它直接为下一章的 self-attention 与 Transformer 铺好了路。

3 Transformer：Attention is All You Need

3.1 章节定位

背景问题

读完第二章，我们已经知道 attention 可以让 decoder 在每一步“挑”encoder 的不同位置看。但 RNN 仍然是骨架：编码端要顺序读完整个源句，解码端要顺序生成。这意味着**长序列训练慢、难以并行**，并且远距离依赖只能通过隐状态间接传递。
那么自然要问：能不能完全不用 RNN，只用 attention 就把序列建模做完？

朴素直觉

2017 年 Vaswani 等人的论文《Attention Is All You Need》给出的回答是：可以。他们提出的 **Transformer** 架构**不含任何 recurrence、也不含卷积**，整套模型只由 attention、线性层、归一化和位置编码拼成。
它带来的直接好处：

- **训练阶段并行**：一句话里所有位置可以同时算，而不是按时间步递推。
- **长依赖一步到位**：任意两个位置之间的“距离”都是 1 次 attention，而不是 $O(n)$ 步隐状态传递。

- **已成为事实标准**：今天几乎所有大语言模型 (GPT/Llama/Qwen/...) 都是 Transformer 的变种。

为了把这一章读“懂”而不是只“看过”，我们围绕 5 个新手最容易卡住的问题来展开：

1. 什么叫**自注意力 (self-attention)**？跟第二章那个 encoder–decoder 之间的 attention 有什么区别？
2. 为什么一个 token 要同时扮演 **Query / Key / Value** 三个角色，不能只用一个向量？
3. 公式里那个 $\sqrt{d_k}$ 是哪来的，能不能不除？
4. **Masked** self-attention 是在 mask 谁、为什么要 mask？
5. **Multi-head** 到底比单头多做了什么？

后面每一节会回答其中一两个。

3.2 自注意力 Self-Attention：让 token 互相看

背景问题

第二章的 attention 是**跨序列**的：decoder 的当前位置当 query，去看 encoder 的所有位置。但编码端自己内部呢？“The animal didn't cross the street because *it* was too tired.” 里面 “it” 到底指 “animal” 还是 “street”？我们希望 encoder 的同一层里，每个词都能看一眼同一句话里的其他词，再决定自己的表示。

朴素直觉

Self-attention 就是把 query、key、value 都来自同一侧的 attention。对一句话里的每个 token，做这样三件事：

- 我把自己变成一个 **Query** 向量 $\boldsymbol{q}$：表示“我想问什么”。
- 我把自己变成一个 **Key** 向量 $\boldsymbol{k}$：表示“别人来找我时，我对外的标签是什么”。
- 我把自己变成一个 **Value** 向量 $\boldsymbol{v}$：表示“如果别人决定关注我，我会贡献什么内容”。

然后让每个 query 跟所有 key 比相似度，得到权重，再用这些权重对所有 value 加权求和，就得到这个 token 的新表示。整句话里所有 token 并行做这件事。

数学上，把整句话堆成矩阵：$Q \in \mathbb{R}^{n\times d_k}$，$K \in \mathbb{R}^{n\times d_k}$，$V \in \mathbb{R}^{n\times d_v}$，其中 n 是序列长度。

Scaled Dot-Product Attention

$$\text{Attention}(Q,K,V) \;=\; \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

其中：

- $QK^\top \in \mathbb{R}^{n\times n}$：第 (i,j) 项就是位置 i 的 query 和位置 j 的 key 的内积，相当于 $\text{score}(\boldsymbol{q}_i,\boldsymbol{k}_j)$。
- 除以 $\sqrt{d_k}$：缩放因子，下面解释。

- softmax 沿**每一行**做：每个 query 对所有 key 的权重和为 1。
- 最后乘 V：用这些权重把 value 加权求和。

备注 3.1 (为什么要除 $\sqrt{d_k}$?). *两个独立、零均值、单位方差的 d_k 维向量做点积，结果的方差大约是 d_k。d_k 一大（例如 64、128），点积的绝对值就会很大；进入 softmax 后，最大那一项会被放得极大，其它几乎归零，softmax 退化成 近似 hard argmax。这时**梯度几乎为 0**，训练走不动。除以 $\sqrt{d_k}$ 把方差压回 ≈ 1，softmax 才有梯度。*

代码 / 工程对应

Q/K/V 是怎么来的？不是凭空写的，而是用三个可学习的投影矩阵从同一份输入 $X \in \mathbb{R}^{n\times d_{\text{model}}}$ 算出来：

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V.$$

也就是说 **"同一个 token 的三种角色" 都是它自己的线性变换**，模型自己学三个角度的 "画像"。

例 3.2 ("it" 该指谁). *继续上面的例子：当 "it" 这个位置的 query 跟 "animal" 的 key 内积大、跟 "street" 的 key 内积小，softmax 之后 "animal" 的权重接近 1，"it" 的新表示就主要由 "animal" 的 value 组成。模型不需要任何递归，一步就把指代关系拉到了眼前。*

3.3 Masked Self-Attention：decoder 不许偷看未来

背景问题

Decoder 在**推理时**是逐词生成的：先吐 y_1，再根据 y_1 吐 y_2… 它物理上看不到未来。
但**训练时**不一样：为了并行，我们会把整句目标序列 $y_1, y_2, \ldots, y_T$ 一次性喂进 decoder。如果直接做 self-attention，位置 t 的 query 会看到 $y_{t+1}, y_{t+2}, \ldots$ 的 key/value，等于直接把答案抄过来——训练 loss 会瞬间归零，但模型啥也没学会。

朴素直觉

解决办法：在 softmax 之前**把 "未来位置" 的分数手动设成** $-\infty$。$\exp(-\infty) = 0$，softmax 之后这些位置的权重自然为 0，就像它们根本不存在一样。这样**并行训练**和**推理时只能看过去**就保持一致。

Causal Mask

设 $S = QK^\top/\sqrt{d_k} \in \mathbb{R}^{n\times n}$ 是未 mask 的分数矩阵，定义因果 mask

$$M_{ij} = \begin{cases} 0, & j \le i \\ -\infty, & j > i \end{cases}$$

则 masked self-attention 为

$$\text{Attention}_{\text{masked}}(Q, K, V) \;=\; \text{softmax}(S + M)\, V.$$

矩阵 $S + M$ 是一个**下三角分数矩阵**，softmax 沿每行做之后上三角全部为 0。

备注 3.3 (和 RNN decoder 的对比)**.** *RNN decoder* 训练一句长度 T 的目标，需要 T **步顺序**前向传播；*masked self-attention* 把这 T 步压成**一次**矩阵乘法，每个位置同时算自己的输出，但通过 *mask* 保证它只用得到自己以前的信息。这就是 *Transformer* 训练比 *RNN* 快很多的关键之一。

3.4 多头注意力 Multi-Head Attention

背景问题

一个词通常跟句子里的别的词存在**多种不同的关系**：主谓一致、修饰、指代、跨从句的并列… 单头 attention 只能学一种 "关注模式"——softmax 那一行决定大头给谁，剩下的就只能分到很小的权重。如果我希望同一句话里既能抓 "it → animal" 的指代，又能抓 "didn't → cross" 的否定关系，单头不够用。

朴素直觉

开**多个并行的注意力头**：每个头有自己独立的 W_Q^i, W_K^i, W_V^i，独立地学一种 "关注什么"。最后把所有头的输出**拼接**起来，再过一个统一的输出投影 W_O 融合一下。

Multi-Head Attention

设有 h 个头，每个头的维度为 $d_k = d_v = d_{\text{model}}/h$。第 i 个头：

$$\text{head}_i \;=\; \text{Attention}\big(QW_Q^i,\; KW_K^i,\; VW_V^i\big).$$

多头注意力的输出为

$$\text{MultiHead}(Q,K,V) \;=\; \text{Concat}(\text{head}_1,\ldots,\text{head}_h)\,W_O.$$

其中 $W_O \in \mathbb{R}^{d_{\text{model}}\times d_{\text{model}}}$ 是输出投影。

代码 / 工程对应

重要的实现细节：多头不会让模型变大。工程上不会真的开 h 套独立的小矩阵，而是：

1. 用**一个**大的线性层一次性算出 Q, K, V，每个的形状是 (B, n, d_{model})；
2. 把最后一维 **reshape** 成 $(h, d_{\text{model}}/h)$，再 **transpose** 让 h 变成 batch 轴；
3. 形状变成 (B, h, n, d_k)，然后照常做 scaled dot-product，等价于 h 个头**并行**计算；
4. 算完再 transpose 回去 reshape 拼接，过 W_O。

所以同样的 d_{model} 下，"单头" 和 "多头" 的参数量是**一样的**，只是把同一份参数切成几份去看不同关系。

下面是一段 PyTorch 风格的伪代码，演示 reshape + transpose 这个技巧：

```
# x: (B, n, d_model)
qkv = self.qkv_proj(x) # (B, n, 3 * d_model)
q, k, v = qkv.chunk(3, dim=-1) # each: (B, n, d_model)

# split heads: d_model -> (h, d_k)
```

```
def split(t):
    return t.view(B, n, h, d_k).transpose(1, 2) # (B, h, n, d_k)

q, k, v = split(q), split(k), split(v)

scores = (q @ k.transpose(-2, -1)) / math.sqrt(d_k) # (B, h, n, n)
if causal_mask is not None:
    scores = scores + causal_mask # -inf on j > i
attn = scores.softmax(dim=-1) @ v # (B, h, n, d_k)

# merge heads back: (B, h, n, d_k) -> (B, n, d_model)
out = attn.transpose(1, 2).contiguous().view(B, n, d_model)
out = self.o_proj(out) # (B, n, d_model)
```

3.5 Transformer 模型架构

到这里 attention 这块的零件都齐了。剩下的事是把它们和几个 “脚手架” 组件拼成一个完整的 encoder/decoder 模型。

3.5.1 Feed-Forward Block (FFN)

背景问题

Self-attention 的工作是 “**看别的位置、把信息搬过来**”。但搬完之后，这个位置自己还需要一步**非线性的处理**，把刚刚收集到的混合信息消化成下一层能用的特征。如果只堆 attention，没有非线性变换，整个模型在线性能力上会很受限。

朴素直觉

在每一层 attention 之后，对**每个位置独立**地过一个 2 层 MLP：先升维、做 ReLU、再降回原维度。位置之间不交互——因为 “跨位置交互” 已经由 attention 负责了，FFN 负责 “**各自消化**”。

Position-wise Feed-Forward

$$\mathrm{FFN}(x) \;=\; \max(0,\; xW_1 + b_1)\, W_2 + b_2.$$

通常 $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$、$W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$，其中 d_{ff} 一般取 $4d_{\text{model}}$。“per-position” 的意思是：同一个 W_1, W_2 应用到序列里每个 token，token 之间不串。

3.5.2 残差连接 Residual Connections

朴素直觉

Transformer 把 attention 和 FFN 都包进同一个模板里：

$$\text{output} \;=\; \mathrm{LayerNorm}\big(x + \mathrm{Sublayer}(x)\big).$$

这个**加法**就是 “Add & Norm” 里的 “Add”，也就是 ResNet 那种残差连接：让信号有一条 “**直通车**” 可以绕过子层。

好处：

- 梯度更容易传到底层，可以放心堆几十层。
- 子层只需要学“在原表示上加什么修正”，比从零学整张表示容易得多。

每一个 attention 子块和每一个 FFN 子块后面都加这种残差连接。

3.5.3 层归一化 Layer Normalization

背景问题

深层网络里激活值的尺度容易飘——某一层突然很大或很小，会让训练不稳定。但 **BatchNorm** 在变长序列上不太合适：不同样本长度不同，按 batch 统计意义不明确，而且**推理时一句一句来**，根本没有 batch 统计。

朴素直觉

换成 **LayerNorm**：对**每一个 token 自己的特征向量**做归一化，也就是沿着 d_{model} 这一维算均值和方差，跟 batch 大小、序列长度都无关。

Layer Normalization

对一个 token 向量 $x \in \mathbb{R}^{d_{\text{model}}}$，

$$\mu = \frac{1}{d}\sum_j x_j, \qquad \sigma = \sqrt{\frac{1}{d}\sum_j (x_j - \mu)^2 + \varepsilon},$$

$$\mathrm{LN}(x) \;=\; \frac{x-\mu}{\sigma} \odot \mathit{scale} \;+\; \mathit{bias},$$

其中 $\mathit{scale}, \mathit{bias} \in \mathbb{R}^{d_{\text{model}}}$ 是可学习参数。

备注 3.4. *LayerNorm 不依赖 batch 维 ⇒ 对变长序列友好；训练和推理行为完全一致，不需要像 BN 那样维护 running mean/var。这也是 NLP 模型几乎清一色用 LayerNorm 而不是 BatchNorm 的原因。*

3.5.4 位置编码 Positional Encoding

背景问题

Self-attention 的公式 $\mathrm{softmax}(QK^\top/\sqrt{d_k})V$ 里**没有任何关于位置的信息**：如果你把输入 token 顺序打乱，输出也只是相应被打乱——模型本身是**置换等变**的。但语言显然不是词袋，“狗咬人”和“人咬狗”意思不同，模型必须知道每个 token 在序列中的位置。

朴素直觉

最简单的做法：给每个位置 pos 配一个“位置向量” $\mathrm{PE}_{pos} \in \mathbb{R}^{d_{\text{model}}}$，直接**加到 token embedding 上**：

$$\tilde{x}_{pos} = \mathrm{Embed}(\mathrm{token}_{pos}) + \mathrm{PE}_{pos}.$$

然后再送入 self-attention。这样同一个词在不同位置就有不同的输入向量，模型自然能区分顺序。

Sinusoidal Positional Encoding（原论文）

$$\mathrm{PE}_{pos,\,2i} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right),$$
$$\mathrm{PE}_{pos,\,2i+1} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right).$$

偶数维放 sin，奇数维放 cos，频率沿维度按几何级数变化：低维高频、高维低频。

备注 3.5 (要不要学位置？)**.** *也可以把 PE 改成**可学习的位置 embedding**（GPT 系就是这么做的）。正弦版本的卖点是：它**不依赖训练长度**，理论上对超过训练时见过的序列长度也能给出有定义的位置向量；学习版本表达力更强，但只在训练长度范围内有意义。两者在常规长度上效果差不多。*

3.5.5 把整体拼起来

万事俱备。一个完整的 Transformer 长这样：

Encoder（共 N 层，原论文 $N = 6$），每层：

1. Self-Attention → Add & Norm
2. FFN → Add & Norm

Decoder（共 N 层），每层：

1. **Masked** Self-Attention → Add & Norm （只看自己前面已生成的位置）
2. **Cross-Attention** → Add & Norm （Q 来自 *decoder*，K/V 来自 *encoder* 顶层输出）
3. FFN → Add & Norm

输出头：decoder 最后一层 → Linear → softmax over vocab，逐位置预测下一个 token。

记住这一点

Cross-attention 是什么？它就是第二章 Bahdanau / Luong attention 在 Transformer 里的“**再实现**”：还是一样的 $\mathrm{softmax}(QK^\top/\sqrt{d_k})V$，只是 Q 来自 *decoder* 当前层（“我现在想要什么信息”），而 K, V 来自 *encoder* 的最终输出（“源句子里有哪些信息可拿”）。所以原本 RNN 时代的 “encoder–decoder attention” 现在只是 Transformer 里**众多 attention 之一**，机制完全统一。

3.6 这一章真正该记住的

1. **纯 attention 就够了**：Transformer 不用 RNN、不用卷积，完全靠 attention + 线性层 + 归一化，证明了 “recurrence” 不是序列建模的必要条件。
2. **并行训练**是它真正的杀手锏：一句话所有位置同时算，训练速度比 RNN 高得多。
3. **Scaled dot-product attention**：核心公式 $\mathrm{softmax}(QK^\top/\sqrt{d_k})V$，$\sqrt{d_k}$ 是为了让 softmax 不饱和。
4. **Masked self-attention** 让 decoder 在并行训练时仍然只能看过去，把 “推理时逐词” 和 “训练时一次性” 统一起来。

5. **Multi-head** 不是变多变大，而是把同样大小的表示**切成几份**，并行学多种关注模式，再用 W_O 融合。

6. **四件脚手架**缺一不可：FFN 做位置内的非线性消化，残差连接保证深层可训，LayerNorm 稳住激活尺度，位置编码补回 attention 缺失的顺序信息。

7. **Encoder = self-attn + FFN** $\times N$**; Decoder = masked self-attn + cross-attn + FFN** $\times N$。 Encoder–decoder 之间的桥梁就是 cross-attention，本质和第二章那套 attention 是同一回事。

4 子词切分：字节对编码 BPE

4.1 章节定位与零基础该问的问题

前三章我们一直假设 "词表是给定的"：模型有一张固定的词典 V，输入输出都从里面选词。但真实文本里永远会出现训练时没见过的拼写、人名、专业术语、罕见词。本章讨论的就是**在固定词表的限制下，如何让模型仍然能处理 "没见过的词"**。答案是**子词（subword）切分**，最常见的实现就是 BPE。

背景问题

[零基础读者该追问的四个问题]

1. 什么叫**固定词表问题（fixed vocabulary problem）**？模型为什么不能临时加词？
2. 什么是 **UNK**？为什么它对翻译这种任务是灾难？
3. BPE 的**训练阶段**到底在做什么？它学出来的是 "词典" 还是 "规则"？
4. BPE 的**推理阶段**又在做什么？面对一个全新的词，它怎么把它切开？

4.2 固定词表问题

背景问题

[问题] 神经网络的输入层 / 输出层维度在搭建时就已经写死：输入 embedding 矩阵是 $|V| \times d$，输出 softmax 是 $d \times |V|$。$|V|$ 一旦定下来，模型就只认识这 $|V|$ 个 token。那遇到词表外的词怎么办？

朴素直觉

[朴素直觉] 最朴素的办法有两端：

- **词级（word-level）**：每个英文单词当一个 token。优点是 "一个词一个意思" 很自然；缺点是英文 + 形态变化 + 人名 + URL 几乎是无穷的，词表 50k 也兜不住，剩下的全部塞进一个特殊 token `<unk>`。一旦句子里出现 `<unk>`，翻译模型就只能输出 `<unk>`，相当于直接放弃。
- **字符级（char-level）**：把句子拆成一个个字符。词表只有几十到几千，永远不会 OOV；但代价是序列变得极长（一个 20 字母的英文词 → 20 个 token），注意力开销 $O(n^2)$ 爆炸，

而且语义被打散到单个字母上，模型学得慢。

中间路线很自然：**常见词保持完整，罕见词切成更小的、训练时见过的子片段**。这就是子词。

备注 4.1. *BPE 被引入 NMT 的工作是 Sennrich, Haddow & Birch (2016)*，Neural Machine Translation of Rare Words with Subword Units。*原本 BPE 是一种数据压缩算法，他们把它“反过来用”：不是为了压缩字节，而是为了**学一张折中粒度的词表**。*

4.3 BPE 算法的两个阶段

BPE 的核心只有一张东西——**有序合并表（merge table）**。训练阶段**学这张表**，推理阶段**按这张表切词**。

4.3.1 训练阶段：学合并规则

BPE 训练循环

初始化：

- 词表 V_0 = 语料里出现过的**所有单字符**（外加一个词尾标记，例如 `</w>`）；
- 合并表 $M = \emptyset$；
- 把每个单词预先切成字符序列，例如 `cat</w>` → `c a t </w>`。

重复 N 次（N 是想要的合并次数，决定最终词表大小）：

1. 在整个语料上数所有**相邻符号对**的频率 $\mathrm{count}(a, b)$；
2. 取频率最高的那一对 $(a^\star, b^\star)$；
3. 把规则 $a^\star + b^\star \to a^\star b^\star$ 追加到合并表 M；把新 token $a^\star b^\star$ 加进词表 V；
4. 在所有已切分的词里，把出现的 $a^\star, b^\star$ 相邻处合并。

备注 4.2 (为什么需要词尾标记)**.** *如果不加 `</w>`（或 SentencePiece 用的 `_`）这种边界符号，合并 `e+r` 会同时影响 `lower` 里的 `er` 和 `her`。加了边界符号之后，`er</w>` 和 `er`（词中间的）就是两个不同符号，保证一次合并只引入一个新 token。*

代码 / 工程对应

[实现上的小技巧] 真正实现时不会逐字符扫语料，而是：

- 先把语料压成 {单词 → 词频} 的字典；
- 数“相邻符号对”时按词频加权，例如 `cat</w>` 出现 4 次，那么对 `(c,a)`、`(a,t)`、`(t,</w>)` 的贡献都各 +4；
- 这样每轮合并的复杂度只跟**不同单词的数量**有关，而不是语料的总长度。

例 4.3 (一个迷你语料)**.** *设语料的单词频次表为*

$$\mathtt{cat}\times 4,\ \mathtt{mat}\times 5,\ \mathtt{mats}\times 2,\ \mathtt{mate}\times 3,\ \mathtt{ate}\times 3,\ \mathtt{eat}\times 2.$$

取 $N = 5$ 次合并。初始词表是字符 $\{a, c, e, m, s, t, \texttt{</w>}\}$，每个词都切成字符。执行训练循环时，最频繁的相邻对会按下面这种顺序被选出：

1. `a+t` 出现在 `cat`, `mat`, `mats`, `mate`, `ate`, `eat`，是最频繁的对，合并 ⇒ 新 *token* `at`；
2. 接下来 `m+at`（来自 *mat* / *mats* / *mate*）频率领先，合并 ⇒ `mat`；
3. 再下来 `at+</w>`（来自原本的 *cat* / *mat* / *ate*）合并 ⇒ `at</w>`；
4. `c+at</w>` 合并 ⇒ `cat</w>`；
5. `e+at`（来自 *eat*）或者 `mat+e`（来自 *mate*）二选一被选出。

五次合并后，常见整词如 `cat</w>`, `mat` 已经成为单 *token*，而 `mate`、`mats` 之类则被表达成 *2–3* 个子词的组合。精确顺序取决于实现的 *tie-breaking* 规则，但**大方向一定是"先合并最频繁的对"**。

4.3.2 推理阶段：按合并表切新词

BPE 推理（编码）

对一个新词 w：

1. 先把它切成字符序列（带词尾标记）；
2. 在当前序列上，找出**所有可能的相邻合并**，其中"可能"指对应的规则在合并表 M 中存在；
3. 选**合并表里位置最靠前**的那条规则（位置越靠前 = 训练时越早被合并 = 越频繁），应用它；
4. 重复 2–3 步，直到没有任何可应用的规则为止；
5. 剩下的符号序列就是该词的 BPE 子词切分。

朴素直觉

[关键：合并表是有序的] 训练时第 1 条规则学出来的 pair 一定比第 100 条更频繁。推理时按这个顺序贪心地合并，相当于优先把"常见整块"凑出来。所以同一个词在不同位置只会被切成同一种结果，BPE 的编码是**确定性**的。

例 4.4 (已见词 vs. 未见词)**.** 假设训练后合并表里靠前的几条是 `a+t`, `m+at`, `at+</w>`, `c+at</w>`, `e+at`。

- 输入 `cat`：字符化为 `c a t </w>` → 合并 `at` → `c at </w>` → 合并 `at</w>` → `c at</w>` → 合并 `cat</w>` → 单 *token* `cat</w>`。
  常见词 = 一个 token。
- 输入 `matador`（训练时没见过）：可能的子词切分是 `mat a d o r </w>` ——`mat` 来自合并表，后面只能停在字符级。罕见词 = 一小串子词，但仍然没有 `<unk>`。

4.4 一些值得知道的工程细节

代码 / 工程对应

[实现家族]

- **原始 BPE** (subword-nmt)：依赖语言级别的预分词（按空格切词），在每个词内部跑 BPE，使用 `</w>` 标记词尾。
- **SentencePiece**：把空格也当成普通符号 _（U+2581），不依赖任何语言相关的预分词，因此天然支持中文/日文这种没有空格的语言。GPT-2/3、LLaMA、Qwen、Gemma 等大模型基本都用 SentencePiece 风格的 BPE。
- **YouTokenToMe / tiktoken / tokenizers (HF)**：对训练 / 编码做了大量工程优化，编码速度可以到数十 MB/s。

备注 4.5 (词表大小的经验值)**.** 小任务的 *BPE* 词表通常 $4k \sim 16k$；*NMT* 经典设置 $32k$；现代大模型 $50k \sim 200k$（多语言往大走）。词表越大，平均序列越短，但 *embedding* 矩阵越占显存。

备注 4.6 (同家族的近亲)**.**

- ***WordPiece*** (*BERT*)：思路非常像 *BPE*，但选择合并对的准则不是“频率最高”，而是使语言模型似然增加最多。
- ***Unigram LM*** (*SentencePiece* 默认选项之一)：反过来——先放一个大词表，再用 *EM* 慢慢删除“最没用”的子词。
- ***BPE-dropout***：训练时以一定概率跳过某些合并，于是同一个词会被随机切成多种合理子词序列，相当于一种数据增强，对低资源任务很有用。

4.5 这一章真正该记住的

记住这一点

[Take-aways]

1. 神经模型词表 V 是固定的，词级切分必然遇到 OOV → `<unk>`，对生成任务尤其致命。
2. 字符级永远没有 OOV，但序列太长、语义被打散；**子词是粒度上的折中**。
3. BPE 的本质是一张**有序的合并表**：训练阶段按“相邻对频率最高”不断合并，逐条学出来。
4. 推理阶段按这张表**从前往后贪心**地把字符序列合并起来；常见词收敛成一个 token，罕见词留成几段。
5. 几乎所有现代 LLM 用的 tokenizer 都是 BPE 或它的近亲（WordPiece / Unigram LM），理解 BPE 就理解了它们 80% 的行为。

5 分析与可解释性 (Analysis and Interpretability)

5.1 章节定位 + 零基础该问的问题

前面几章我们一路堆模型：先有 seq2seq，再加 attention，再加 multi-head，每一步都伴随着一个“直觉理由”——比如“注意力应该能学到对齐”，“多头应该能学到不同种类的关系”。但这些

都只是设计动机，并不等于模型真的照做了。这一章要做的事是：**打开训练好的模型，看看里面到底发生了什么**。

背景问题

零基础读者会问的三个问题

1. 多头注意力的每个 head 真的在干不同的事吗？还是大家学的都差不多？
2. 既然这样，能不能干脆把一些“没用”的 head 裁掉？
3. Encoder 内部的向量表示，到底编码了什么语言学信息（词性？形态？句法？）？

下文以两条研究线分别回应：第一条来自 Voita et al. (ACL 2019, *Analyzing Multi-Head Self-Attention*), 第二条来自 Belinkov et al. (2017) 等围绕 NMT encoder 的 probing 工作。

5.2 多头自注意力的头到底在干嘛？

5.2.1 重要的 head 是可解释的

Voita et al. 训练好一个 Transformer NMT 模型后，逐个 head 分析其行为，得到一个非常干净的结论：**真正“干活”的只是少数 head**，而这些 head 的角色可以被人读懂。

朴素直觉

不是每个 head 都是平等的。一个训练好的多头自注意力，更像是“几个专家 + 一堆陪跑”。

被识别出的“专家”大体可以归成三类：

(a) 位置型 head (positional heads) 几乎只看相邻位置。Encoder 中典型地有 2–3 个“看前一个 token”的 head，以及 ~2 个“看后一个 token”的 head。这相当于模型自己学出了一个轻量级的局部卷积。

(b) 句法型 head (syntactic heads) 会跟踪“主语 → 动词”“动词 → 宾语”这类语法关系。这种行为在多个不同语言对的模型里都能观察到，说明它不是某种偶然，而是 multi-head 自然倾向于做的事。

(c) 稀有词型 head (rare-tokens head) 在第一层往往会出现一个特殊的 head：它把注意力集中到句子里最稀有的 token 上。一个合理的解释是，模型在“锚定”那些信息量最大的稀有线索；同时这个现象也带有一点轻微的过拟合气味（稀有 token 在训练集里本来就更具区分度）。

这个结果反过来验证了 multi-head 的归纳偏置：**不同 head 确实会学到不同的东西**，并不是简单地把同一种关系算了 h 遍。

5.2.2 大多数 head 可以被剪掉

同一篇论文进一步做了一件激进的事：给每个 head 套上一个可学习的“门”(gate)，通过稀疏正则把不重要的 head 推向 0，然后看模型质量随被裁 head 数量的下降。

观察到两个互相印证的现象：

1. **很大一部分 head 可以被裁掉**，翻译质量几乎不掉。典型设置下，只剩下 1/4 ∼ 1/3 的 head，BLEU 仍然接近原模型。
2. **留下来的 head 恰好就是上面归类的那些“专家”**（位置型、句法型、稀有词型）。

但有一个看起来反直觉、却非常重要的细节：

记住这一点

这不意味着可以一开始就训练一个 head 数很少的模型。直接用少头训练，质量明显更差。多头在训练时的真正作用更像是一个更大的假设空间：让模型有机会“尝试”各种行为模式，最后挑出有用的那几个。剪枝是事后压缩，而不是先验设计。

5.3 Probing：表示里到底编码了什么？

第二个问题指向 encoder 输出的那一堆向量：它们里面除了“能用来翻译”，是否还藏着具体的语言学信息？衡量这件事的标准工具叫 **probing**（探针）。

5.3.1 Probing 的方法论

Probing 的标准流程

1. **冻结**已经训练好的 NMT encoder（参数不再更新）。
2. 把一批带标注的数据（例如每个词都有词性标签）喂进去，在指定层取出每个词的向量表示 h_i。
3. 在这些冻结的 h_i 之上，训练一个小分类器（通常是线性层或浅层 MLP）去预测目标标签 y_i。
4. 用这个小分类器的准确率作为代理指标，近似回答“h_i 里有多少关于 y_i 的信息”。

备注 5.1. 一个常被忽视的注意点：*probing* 的准确率本身并不直接等于“表示里编码了多少信息”，因为探针自己就有一定学习能力，甚至能从随机特征里学出一些东西。严格的 *probing* 研究通常会做对照实验（比如随机初始化的 *encoder*、打乱标签的对照、或控制探针容量）。本节只用其结论性观察，对方法论细节不再展开。

5.3.2 NMT encoder 学到了什么形态学？

Belinkov et al. (2017) 用 probing 方法系统研究了 NMT encoder。两个值得记住的结论：

(a) 词性 (POS) 与层数的关系 拿 encoder 各层的表示去 probe POS 标注：

- 训练后的 encoder 各层都比原始词嵌入显著更好；
- 但有趣的是，**第 1 层的 POS 探针准确率往往高于第 2 层**。

对应的解释假设是：第 1 层更接近“词级结构”（词性、形态等表层属性），第 2 层开始向更抽象、更语义化的方向移动，反而把表层语法信息“压”了下去。这与“层数越深 = 越好”的朴素直觉是矛盾的。

(b) 目标语言的反向影响 固定 source 语言（比如阿拉伯语），变化 target 语言，再 probe source 端形态信息：

朴素直觉

直觉上，target 越“强”，encoder 越要努力工作，所以 encoder 的表示应该越丰富。

实际观察到的是这样一个稍微反直觉的结果：

> **Target 语言形态越贫乏（如英语），encoder 对 source 形态的编码反而越好；target 语言形态越丰富（如希伯来语、德语），encoder 反而编码得更弱。**

一个解释方式是：当 target 自己有丰富的屈折变化时，模型可以“偷懒”，让大部分形态信息直接从 target 端的解码器/词表那里产生；而 target 一旦贫乏（英语几乎没有屈折），encoder 就不得不在自己这一侧把 source 的形态彻底搞清楚，否则解码器没法做出正确翻译。

5.4 这一章真正该记住的

记住这一点

1. **头不平等**：训练好的多头注意力里，只有一小部分 head 真正承担主要工作。
2. **角色可解释**：重要的 head 倾向于扮演位置型、句法型、稀有词型这几种角色，说明 multi-head 的归纳偏置确实在起作用。
3. **剪枝可行，但前提是先有多头**：可以事后裁掉大部分 head 而几乎不掉点；但直接用少头从头训练效果会明显更差——多头主要是训练阶段的假设空间。
4. **Probing 是“表示里有什么”的标准工具**：冻结 encoder，在表示之上训练浅层分类器，用准确率近似信息量；但要小心探针自身能力带来的偏差。
5. **Target 越弱，encoder 越强**：当 target 语言形态贫乏时，NMT encoder 反而会把 source 的形态信息编码得更彻底。

致谢与原文

本文所有图示思路、公式与例子来源于 Lena Voita 的公开课程页面：https://lena-voita.github.io/nlp_course/seq2seq_and_attention.html。本中文讲义只是按零基础读者的视角做了重写与展开，以便把原页面的「先看图、再看公式」体验补回到纯文本场景。